Лабораторная работа №12

Код

asm_client.asm

```
nasm
section .data
    sock_addr:
        dw 2
        dw 0x901F
        dd 0x0100007F
        dq 0
    sock_addr_len equ $ - sock_addr

    msg     db "Hello from clean NASM assembly code!", 10
    msg_len equ $ - msg

section .text
    global _start

_start:
    mov rax, 41
    mov rdi, 2
    mov rsi, 1
    mov rdx, 0
    syscall

    mov r12, rax

    mov rax, 42
    mov rdi, r12
    mov rsi, sock_addr
    mov rdx, sock_addr_len
    syscall

    mov rax, 1
    mov rdi, r12
    mov rsi, msg
    mov rdx, msg_len
    syscall

    mov rax, 60
```

```
    xor rdi, rdi
    syscall
```

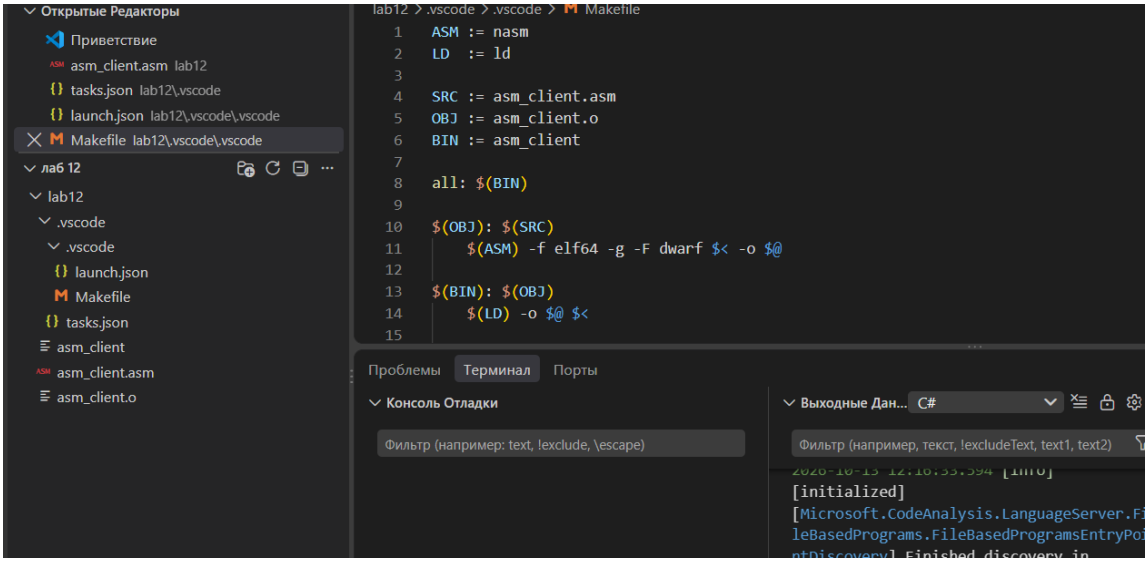
Сборка и запуск

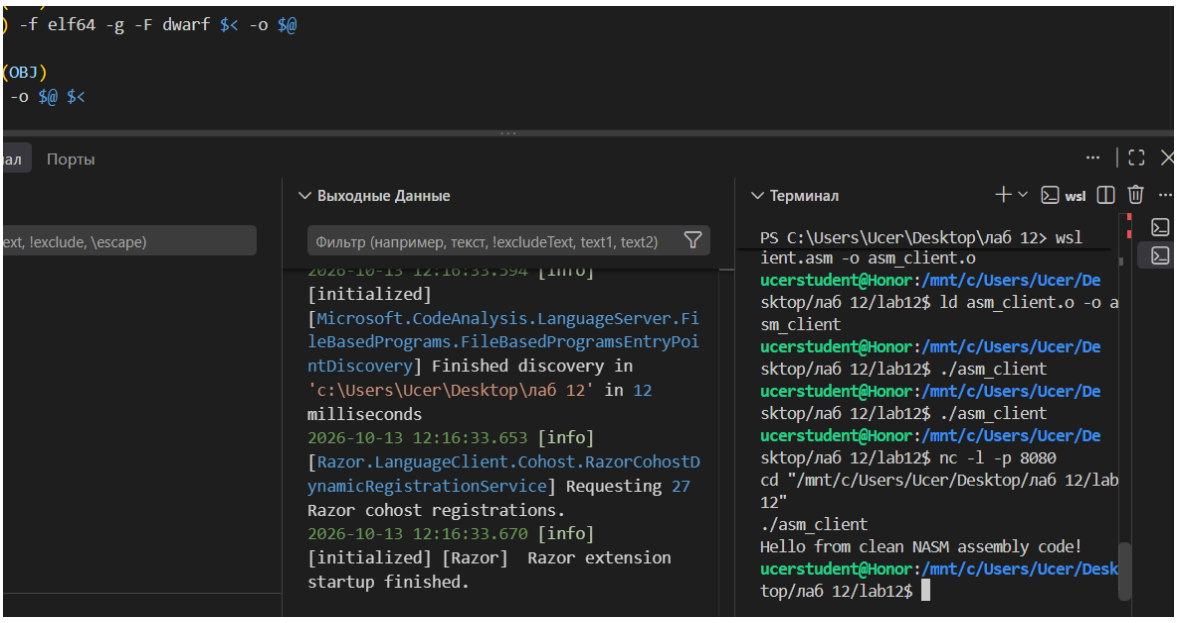
```
bash
nasm -f elf64 asm_client.asm -o asm_client.o
ld asm_client.o -o asm_client
```

```
bash
nc -l -p 8080
```

```
bash
./asm_client
```

Скриншоты





Контрольные вопросы

1. Порядок байт (endianness)

x86-64 — Little-Endian: младший байт лежит первым. Сеть (TCP/IP) — Big-Endian: старший байт первым. Поэтому перед отправкой порт и IP переворачиваем (htons/htonl), иначе сервер прочитает 8080 как мусор.

2. Регистры при ошибке Connect в GDB

Системный вызов возвращает результат в регистре **rax**. Если ошибка — rax отрицательный (например, -111 = ECONNREFUSED, «сервер не запущен»). Linux сигнализирует об ошибке через флаг переноса (CF) и отрицательное значение в rax, а не через отдельный регистр.

3. Цепочка send() в Python

`send()` (Python) → `socket.send` (C-библиотека) → системный вызов **write/sendto** → инструкция **syscall** → переход Ring 3 → Ring 0 → ядро кладёт номер вызова и аргументы в регистры (rax, rdi, rsi, rdx) → драйвер сетевой карты → байты уходят в сеть.